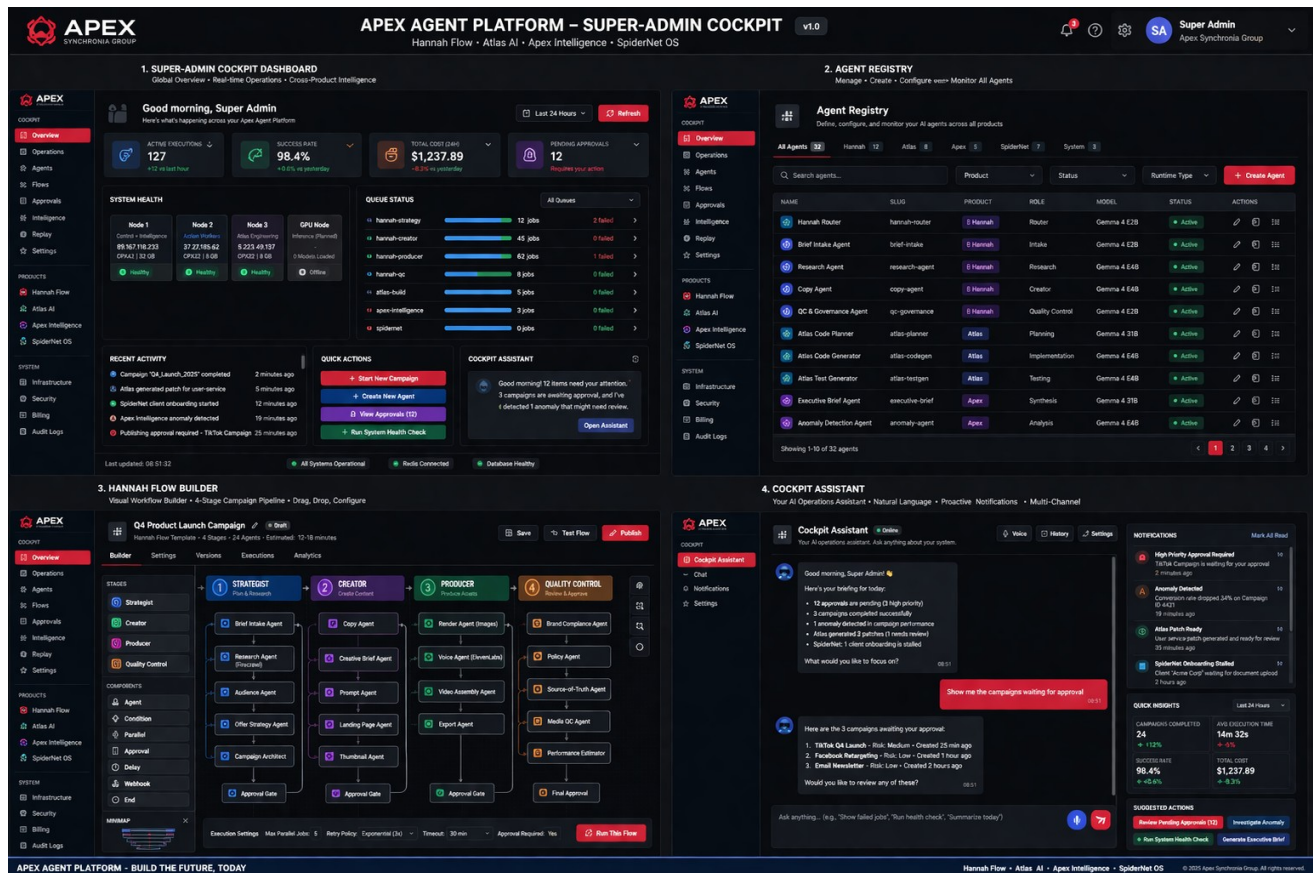


# APEX AGENT PLATFORM

## Super-Admin Cockpit, Hannah Flow, Atlas, Apex Intelligence, and SpiderNet OS

### Integrated Implementation Document

*Deployment-grade architecture, UI and data model, queue strategy, CRUD workflows, drag-and-drop schema, assistant logic, and phased delivery roadmap*



Prepared for Apex Synchronia Group. This document consolidates the architectural, operational, UI, governance, and implementation decisions developed in the working session.

## Document Map

1. Executive summary and target operating model
2. Current node inventory and exact target deployment layout
3. Service boundaries and Laravel vs Python ownership
4. Queue map and workload segregation
5. Super-admin cockpit operating model
6. Information architecture and screen-by-screen UI specification
7. Exact UI database migrations and schema notes
8. Controllers, routes, authorization, and CRUD flows
9. Livewire/Vue component map and state model
10. Agent registry CRUD system
11. Hannah Flow builder drag-and-drop schema and execution mapping
12. Cockpit assistant backend logic and notification system
13. Redis/Horizon assignments, database plan, environment matrix
14. Phased build order for Hannah, Atlas, Apex Intelligence, and SpiderNet
15. Security, approvals, event replay, testing, and acceptance criteria
16. Source references and implementation notes

This specification is intentionally long-form. It is designed to function as a build document, a technical alignment document, and a governance document. The audience is the founder/super admin, product and engineering leadership, backend engineers, frontend engineers, DevOps engineers, and QA reviewers.

## 1. Executive Summary and Target Operating Model

Apex should be built as a four-plane agent platform with one dominant application operating on top of it first: Hannah Flow. Hannah Flow is the first production product because it is closest to your current Boa Flow work and has the clearest revenue and operational payoff. Atlas is the internal engineering plane built second, Apex Intelligence is the supervisory plane built third, and SpiderNet OS is the multi-tenant commercialization layer built fourth.

The key design principle is simple: the user experiences one click, but the system executes a managed graph of specialist agents, bounded tools, explicit approvals, queue-isolated workloads, and event-backed replay. This approach matches OpenAI's practical guidance that manager-pattern orchestration with specialist delegates is usually safer and more maintainable than one monolithic autonomous agent [R1]. It also aligns with Laravel's first-party support for queues, Horizon, notifications, scheduler-driven workflows, and Livewire-powered administrative interfaces [R2][R3][R4][R5].

The three existing Hetzner servers are sufficient for a credible version-one platform if they are assigned clean boundaries. The largest node becomes the Control plus Intelligence plane. The existing Boa AI node becomes the Action plane. The Atlas node becomes the Engineering plane. A

fourth GPU node is added later as the dedicated Inference plane. This is the correct structure because Hetzner’s dedicated-resource cloud positioning is aimed at sustained business workloads, while heavier local AI inference belongs on GPU-oriented infrastructure [R6][R7].

Gemma 4 should be integrated as the model family for local routing, planning, coding, and review once the GPU node exists. Google’s current Gemma documentation highlights improved coding and agentic capabilities, native system prompts, and function-calling support, which makes it a strong fit for internal agent systems as long as execution is still wrapped in strict application-level controls [R8][R9].

NemoClaw and OpenShell are useful as reference security patterns for staged experimentation, especially for sandboxed OpenClaw-style execution, but they should not be treated as your production trust anchor yet. NVIDIA’s own documentation currently marks NemoClaw as alpha / early preview and explicitly warns against production use at this stage [R10][R11].

Firecrawl is beneficial in this architecture and should be included. It belongs on the Action plane as a web intelligence layer for crawling source pages, extracting grounded content, gathering proof points, generating screenshots, and supporting source-of-truth checks during quality control [R12].

## 2. Current Node Inventory and Exact Node-by-Node Deployment Layout

| Node               | IP             | Current Size   | Assigned Plane         | Primary Mission                                                                      |
|--------------------|----------------|----------------|------------------------|--------------------------------------------------------------------------------------|
| Boa-ai-RAG-BOAFlow | 89.167.118.233 | CPX42 / 320 GB | Control + Intelligence | Primary Laravel core, state, dashboards, approvals, event store, KPI rollups         |
| spidernet-boaai    | 37.27.185.62   | CPX22 / 80 GB  | Action                 | Browser, crawl, extract, render, publish prep, connector workers                     |
| spidernet-atlas    | 5.223.49.137   | CPX22 / 80 GB  | Engineering            | Atlas services, repo ingestion, code planning, code/test generation, patch workflows |
| Future GPU node    | TBD            | GPU class      | Inference              | Gemma family serving, embeddings, reranking, inference gateway, fallback routing     |

This section converts the agreed architecture into concrete node responsibilities. The design objective is not theoretical purity; it is operational calm. Each node should have a dominant mission and should not silently absorb unrelated heavy workloads.

## 2.1 Node 1 - Control + Intelligence Plane

- Laravel application core and super-admin cockpit
- Authentication, authorization, RBAC, and tenant/workspace scoping
- Hannah Flow orchestration and durable execution state
- Agent registry, templates, approvals, audit, event store, replay metadata
- Redis, Horizon, scheduler, notifications, dashboard APIs, Apex Intelligence KPI rollups
- Primary relational database: PostgreSQL preferred for the new platform; MySQL possible where continuity with existing BoaFlow code is more valuable

Node 1 must remain clean. It is the source of truth. It should not perform noisy browser automation, document conversion spikes, ffmpeg-intensive composition, or unbounded code execution.

## 2.2 Node 2 - Action Plane

- FastAPI action gateway and worker manager
- Playwright or browser pool
- Firecrawl adapter or Firecrawl self-hosted service layer
- Document processors, screenshot capture, structured web extraction
- Media preprocessing and export packaging workers
- Publisher/connectors for social, CMS, CRM, email, and approved outbound integrations

Node 2 is intentionally noisy. It is where the system touches the outside world, transforms documents, captures pages, and assembles outbound payloads. Failures are expected here; they must be observable but must not destabilize the control plane.

## 2.3 Node 3 - Atlas / Engineering Plane

- Atlas API, repo ingestion, architecture planning, codegen, test generation, static review, patch bundling
- Sandboxed code runners for low-risk execution and validation tasks
- Internal engineering automation only; no direct production deployment authority by default

## 2.4 Future Node 4 - Inference Plane

The GPU node is the next major infrastructure upgrade. Until it exists, model calls should route to external APIs where needed. Once it exists, it becomes the local inference gateway with optional external overflow.

| Task class                                        | Preferred model tier | Reason                                            |
|---------------------------------------------------|----------------------|---------------------------------------------------|
| Fast routing, classification, metadata extraction | Gemma 4 E2B          | Lower latency, cheaper throughput for high-volume |

|                                                                |                                       |                                                                |
|----------------------------------------------------------------|---------------------------------------|----------------------------------------------------------------|
|                                                                |                                       | operational work                                               |
| Campaign strategy, copy generation, standard review            | Gemma 4 E4B                           | Better reasoning at manageable cost                            |
| Hard code architecture, executive synthesis, complex debugging | Gemma 4 26B A4B / 31B or API fallback | High-value deep reasoning workloads benefit from larger models |

### 3. Service Boundaries and Laravel vs Python Ownership

This platform will only remain governable if service boundaries are respected. The most important operational rule is this: Laravel owns workflow truth and business truth. Python owns tool use, compute-heavy execution, and inference. Python services may perform work, but Laravel persists the official state transitions, approvals, and audit history.

#### 3.1 Laravel ownership

- Identity, SSO, role and permission model, workspace and tenant boundaries
- Templates, versions, flow definitions, publish rules, approval rules
- FlowExecutionService and durable execution records
- Agent registry, instruction profiles, guardrail profiles, approval policies, secret scopes
- Super-admin cockpit APIs, dashboard views, queue health summaries, replay console metadata
- Notification routing, approval inbox, event persistence, KPI materialization and summaries

#### 3.2 Python ownership

- Playwright/browser automation, crawling and extraction, document conversion, screenshotting, media assembly
- Atlas repository analysis, code generation helpers, static review, test writing, patch bundling
- Inference gateway and local model serving
- Heavy connectors and worker runtimes that would bloat or destabilize the PHP application

#### 3.3 Contract between layers

1. Laravel creates an execution intent and persists the parent execution, stage, and job records.
2. Laravel dispatches a signed internal job payload onto a queue or internal service endpoint.
3. Python executes the requested work under explicit runtime limits and tool permissions.
4. Python returns structured outputs, metrics, and artifact references.
5. Laravel persists canonical status, approval state, replay metadata, and notifications.

This contract prevents hidden state drift and makes event replay, approvals, and incident analysis practical.

## 4. Queue Map and Workload Segregation

Queues should be grouped by workload class, not by convenience. This matters because copy generation, browser automation, publishing, and Atlas code tasks have radically different runtime, memory, failure, and retry profiles.

| Queue family                                       | Primary node                    | Purpose                                                      |
|----------------------------------------------------|---------------------------------|--------------------------------------------------------------|
| hannah-orchestrator / hannah-stage-control         | Node 1                          | Stage orchestration, joins, retries, approval transitions    |
| hannah-strategy-*                                  | Node 1 + Node 2 + future Node 4 | Brief structuring, web research, offer and audience planning |
| hannah-copy-* / hannah-prompt-*                    | Node 1 + future Node 4          | Copy, prompts, landing-page text, thumbnails                 |
| hannah-render-* / hannah-compose-* / hannah-export | Node 2                          | Image/voice/render assembly and export packaging             |
| hannah-qc-*                                        | Node 1 + Node 2 + future Node 4 | Brand, policy, format, source-truth, ranking                 |
| atlas-*                                            | Node 3                          | Repo ingest, plan, codegen, testgen, review, debug           |
| apex-*                                             | Node 1                          | Executive brief, anomaly detection, KPI rollups              |
| spidernet-*                                        | Mixed later                     | Onboarding, integration, tenant workflow execution           |
| maintenance-*                                      | Node 1                          | Schedules, retries, replay, cleanup, compaction              |

Horizon should supervise only the Laravel-managed queue families. Python workers should use Redis or internal endpoints to consume the queues that correspond to their runtimes, while still reporting completion back into Laravel.

## 5. Super-Admin Cockpit Operating Model

The cockpit is not a dashboard in the lightweight sense. It is the command and governance interface for the entire platform. It needs a super-admin mode, an operator mode, an approver mode, and an auditor mode so the same person can operate under different responsibilities when necessary.

| Mode        | Core powers                                                    | Typical use              |
|-------------|----------------------------------------------------------------|--------------------------|
| Super Admin | Full visibility, policy control, agent control, kill switches, | Founder / platform owner |

|          |                                                                     |                                                       |
|----------|---------------------------------------------------------------------|-------------------------------------------------------|
|          | overrides                                                           |                                                       |
| Operator | Runbooks, retries, queue and worker actions, replay and diagnostics | Operations / engineering response                     |
| Approver | Approve or reject high-impact actions only                          | Publishing, privileged changes, client-facing actions |
| Auditor  | Read-only access to evidence trails and decisions                   | Governance, compliance, postmortems                   |

## 5.1 Cockpit modules

- Overview / operations dashboard
- Agent registry console
- Hannah Flow builder and execution views
- Approvals center
- Apex Intelligence insights and anomaly surfaces
- Incident replay console
- Cockpit assistant panel and notification center
- Infrastructure, billing, security, and audit surfaces

## 5.2 Cockpit assistant role

The cockpit assistant is a dedicated operations agent that consumes the event stream, queue and node health, approval backlog, and performance metrics, then translates that into useful notifications, short briefings, and commandable actions. It should not be just a passive chatbot. It should proactively surface what needs attention, summarize risk, and guide the super admin through corrective action.

# 6. Information Architecture and Screen-by-Screen UI Specification

The UI system below maps directly to the high-fidelity cockpit mockup and to the underlying data model. The recommendation is Laravel plus Livewire for administrative screens, with Vue embedded selectively where canvas-heavy, drag-and-drop, or highly interactive interfaces justify it.

## 6.1 Global layout shell

| Region          | Purpose                                  | Key elements                                                                                      |
|-----------------|------------------------------------------|---------------------------------------------------------------------------------------------------|
| Left navigation | Persistent product and system navigation | Overview, Operations, Agents, Flows, Approvals, Intelligence, Replay, Settings, product switchers |
| Top bar         | Global controls and identity             | Search, environment badge,                                                                        |

|              |                           |                                                               |
|--------------|---------------------------|---------------------------------------------------------------|
|              |                           | notifications, account switch, health indicator               |
| Main canvas  | Primary working area      | Tables, forms, charts, builders, assistants, execution detail |
| Right drawer | Context and notifications | Assistant thread, alerts, approval snippets, quick actions    |

## 6.2 Overview dashboard

- KPI cards: active executions, success rate, real-time cost, pending approvals
- Node health panel: control, action, atlas, inference
- Queue status panel by queue family with failure counts
- Recent activity feed, quick actions, assistant summary card
- Cross-product status: Hannah, Atlas, Apex Intelligence, SpiderNet

## 6.3 Agent registry UI

- Table view filtered by product, role, status, runtime type, provider, model
- Detail drawer or page with configuration, tasks, guardrails, performance, change history
- Bulk actions: pause, activate, assign queue, rotate model, clone, archive
- Create/edit form with validation and explicit scope controls

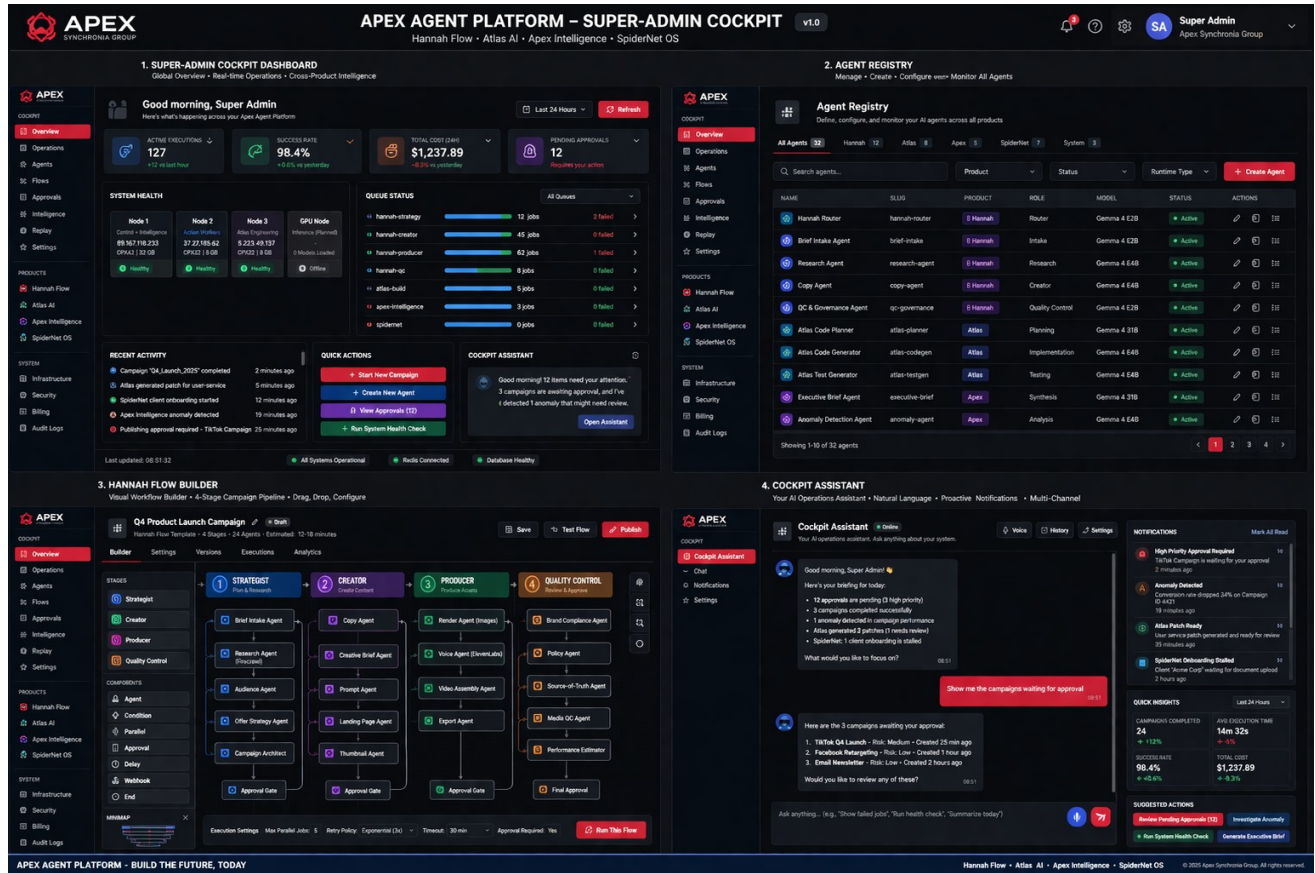
## 6.4 Hannah Flow builder UI

The flow builder is a visual orchestration editor for Hannah templates and flow versions. It must support stage-level topology, drag-and-drop of node types, approval gates, parallel branches, conditions, webhook nodes, and execution settings. The system should maintain a canonical JSON definition in the database and render that definition into the builder canvas.

## 6.5 Cockpit assistant interface

- Chat thread with command suggestions and structured reply cards
- Notification stack with severity, action state, affected product, and deep links
- Quick insights cards: approvals pending, anomalies, failed jobs, success rate trend, total platform cost
- Context-aware command composer: show failed jobs, review approvals, summarize today, run health check

## 6.6 Reference UI mockup



Reference composite mockup showing the intended relationship between the Overview, Agent Registry, Hannah Flow Builder, and Cockpit Assistant surfaces.

## 7. Exact UI Database Migrations and Schema Notes

This section specifies the exact data structures needed to turn the cockpit and the flow tooling into a working Laravel application. Some of the tables extend the earlier BOA Flow tables. Others are new and specifically support UI, permissions, versioning, and conversational assistant behavior.

| Migration                        | Purpose                                   |
|----------------------------------|-------------------------------------------|
| create_ui_navigation_items_table | Configurable navigation and badges        |
| create_roles_permissions_tables  | Role and permission system                |
| create_agent_definitions_table   | Master agent registry                     |
| create_agent_tasks_table         | Agent task catalog                        |
| create_agent_runs_table          | Operational run history                   |
| create_flow_templates_table      | Flow builder templates and versions       |
| create_flow_nodes_table          | Visual node registry per template version |

|                                   |                                             |
|-----------------------------------|---------------------------------------------|
| create_flow_edges_table           | Graph connectivity for drag-and-drop canvas |
| create_approval_policies_table    | Approval requirements and thresholds        |
| create_guardrail_profiles_table   | Brand/policy/output validation profiles     |
| create_instruction_profiles_table | Prompt and instruction versions             |
| create_notifications_table        | Cockpit and assistant notifications         |
| create_assistant_threads_table    | Cockpit assistant conversation threads      |
| create_assistant_messages_table   | Assistant/user messages and payload cards   |
| create_system_nodes_table         | Server inventory and health metadata        |
| create_queue_metrics_table        | Materialized queue and worker metrics       |
| create_replay_sessions_table      | Execution replay requests and outputs       |

## 7.1 Exact migration - agent\_definitions

```
<?php
```

```
Schema::create('agent_definitions', function (Blueprint $table) {
    $table->ulid('id')->primary();
    $table->foreignUlid('tenant_id')->nullable()->index();
    $table->string('slug')->unique();
    $table->string('name');
    $table->string('product_scope'); // hannah, atlas, apex_intelligence, spidernet, shared
    $table->string('role');
    $table->string('runtime_type'); // laravel_managed, python_service, sandboxed_worker, inference_proxy
    $table->string('status')->default('active');
    $table->string('provider')->nullable();
    $table->string('default_model')->nullable();
    $table->json('allowed_job_types')->nullable();
    $table->json('allowed_queues')->nullable();
    $table->json('tool_allowlist')->nullable();
    $table->foreignUlid('approval_policy_id')->nullable()->index();
    $table->foreignUlid('secret_scope_id')->nullable()->index();
    $table->string('tenant_scope_mode')->default('global');
    $table->foreignUlid('cost_budget_policy_id')->nullable()->index();
    $table->unsignedInteger('max_runtime_seconds')->nullable();
    $table->unsignedInteger('max_token_budget')->nullable();
    $table->unsignedInteger('max_artifact_size_mb')->nullable();
    $table->foreignUlid('instruction_profile_id')->nullable()->index();
    $table->foreignUlid('guardrail_profile_id')->nullable()->index();
    $table->foreignUlid('supervisor_agent_id')->nullable()->index();
    $table->foreignUlid('created_by')->nullable()->index();
    $table->foreignUlid('updated_by')->nullable()->index();
});
```

```

    $table->boolean('is_system_agent')->default(false);
    $table->timestamps();
});

```

## 7.2 Exact migration - agent\_tasks

```

Schema::create('agent_tasks', function (Blueprint $table) {
    $table->ulid('id')->primary();
    $table->foreignUlid('agent_definition_id')->constrained('agent_definitions')->cascadeOnDelete();
    $table->string('task_key');
    $table->string('name');
    $table->string('job_type');
    $table->string('queue_name');
    $table->json('input_schema')->nullable();
    $table->json('output_schema')->nullable();
    $table->json('default_payload')->nullable();
    $table->json('runtime_rules')->nullable();
    $table->foreignUlid('retry_profile_id')->nullable()->index();
    $table->string('idempotency_strategy')->nullable();
    $table->boolean('is_active')->default(true);
    $table->timestamps();

    $table->unique(['agent_definition_id', 'task_key']);
});

```

## 7.3 Exact migration - flow builder tables

```

Schema::create('flow_templates', function (Blueprint $table) {
    $table->ulid('id')->primary();
    $table->string('slug')->unique();
    $table->string('name');
    $table->string('product_scope')->default('hannah');
    $table->unsignedInteger('version')->default(1);
    $table->string('status')->default('draft');
    $table->text('description')->nullable();
    $table->json('settings')->nullable();
    $table->json('publish_rules')->nullable();
    $table->foreignUlid('created_by')->nullable()->index();
    $table->foreignUlid('updated_by')->nullable()->index();
    $table->timestamps();
});

```

```

Schema::create('flow_nodes', function (Blueprint $table) {
    $table->ulid('id')->primary();
    $table->foreignUlid('flow_template_id')->constrained('flow_templates')->cascadeOnDelete();
    $table->string('node_key');
    $table->string('node_type'); // agent, condition, approval, parallel, delay, webhook, start, end
    $table->string('stage_name')->nullable(); // strategist, creator, producer, quality_control
    $table->unsignedInteger('x')->default(0);
    $table->unsignedInteger('y')->default(0);
    $table->json('config')->nullable();
    $table->json('ui_meta')->nullable();
    $table->timestamps();
    $table->unique(['flow_template_id', 'node_key']);
});

Schema::create('flow_edges', function (Blueprint $table) {
    $table->ulid('id')->primary();
    $table->foreignUlid('flow_template_id')->constrained('flow_templates')->cascadeOnDelete();
    $table->foreignUlid('from_node_id')->constrained('flow_nodes')->cascadeOnDelete();
    $table->foreignUlid('to_node_id')->constrained('flow_nodes')->cascadeOnDelete();
    $table->string('edge_type')->default('default'); // default, success, failure, conditional
    $table->json('conditions')->nullable();
    $table->timestamps();
});

```

## 7.4 Exact migration - cockpit assistant

```

Schema::create('assistant_threads', function (Blueprint $table) {
    $table->ulid('id')->primary();
    $table->foreignUlid('tenant_id')->nullable()->index();
    $table->foreignUlid('user_id')->nullable()->index();
    $table->string('context_scope')->default('super_admin_cockpit');
    $table->string('status')->default('active');
    $table->json('context_snapshot')->nullable();
    $table->timestamps();
});

Schema::create('assistant_messages', function (Blueprint $table) {
    $table->ulid('id')->primary();
    $table->foreignUlid('thread_id')->constrained('assistant_threads')->cascadeOnDelete();
    $table->string('sender_type'); // user, assistant, system
    $table->text('content');
    $table->json('structured_payload')->nullable();
    $table->json('references')->nullable();
    $table->timestamps();
});

```

```
Schema::create('notifications', function (Blueprint $table) {
    $table->ulid('id')->primary();
    $table->foreignUlid('tenant_id')->nullable()->index();
    $table->foreignUlid('user_id')->nullable()->index();
    $table->string('source_type'); // system, assistant, approval, anomaly, execution
    $table->string('severity')->default('info');
    $table->string('title');
    $table->text('message');
    $table->json('context_json')->nullable();
    $table->boolean('action_required')->default(false);
    $table->string('status')->default('unread');
    $table->timestamp('read_at')->nullable();
    $table->timestamp('acted_at')->nullable();
    $table->timestamps();
});
```

## 7.5 Supporting schema notes

The schema above should be implemented alongside the earlier execution, stage, job, artifact, event, guardrail, and publish tables. Together they create a single coherent operating model rather than a fragmented admin surface.

## 8. Laravel Controllers, Routes, Authorization, and CRUD Flows

The controllers below are intentionally organized by product and function. Keep the route structure predictable. Every surface in the super-admin cockpit should have a controller responsible for its server-rendered state, and an API or Livewire endpoint structure responsible for mutations and asynchronous refreshes.

| Controller                 | Responsibility                                                                |
|----------------------------|-------------------------------------------------------------------------------|
| CockpitDashboardController | Overview cards, node health, queue health, recent activity, assistant summary |
| AgentRegistryController    | List/filter/show agents and create/update/archive                             |
| AgentTaskController        | CRUD for tasks attached to an agent                                           |
| AgentPerformanceController | Run history and agent KPIs                                                    |
| FlowTemplateController     | CRUD for templates and versions                                               |
| FlowBuilderController      | Canvas payload, node/edge mutations, validation                               |
| ApprovalCenterController   | List/approve/reject/send-back high-impact actions                             |
| ReplayConsoleController    | Execution replay and restart-from-stage requests                              |

|                            |                                                                   |
|----------------------------|-------------------------------------------------------------------|
| AssistantController        | Thread CRUD, send message, fetch responses, quick commands        |
| NotificationController     | Inbox, mark-as-read, act-on-notification                          |
| InfrastructureController   | Node status, worker status, env overview, secrets exposure checks |
| ApexIntelligenceController | Executive summaries, anomalies, recommendations, product rollups  |

## 8.1 Route map

```

Route::middleware(['auth', 'verified'])->prefix('cockpit')->name('cockpit.')->group(function () {
    Route::get('/', [CockpitDashboardController::class, 'index'])->name('dashboard');

    Route::resource('agents', AgentRegistryController::class);
    Route::resource('agents.tasks', AgentTaskController::class)->shallow();
    Route::get('agents/{agent}/performance', [AgentPerformanceController::class, 'show'])->
    >name('agents.performance');

    Route::resource('flows', FlowTemplateController::class);
    Route::get('flows/{flow}/builder', [FlowBuilderController::class, 'show'])->name('flows.builder');
    Route::post('flows/{flow}/nodes', [FlowBuilderController::class, 'storeNode'])->name('flows.nodes.store');
    Route::patch('flows/{flow}/nodes/{node}', [FlowBuilderController::class, 'updateNode'])->
    >name('flows.nodes.update');
    Route::delete('flows/{flow}/nodes/{node}', [FlowBuilderController::class, 'destroyNode'])->
    >name('flows.nodes.destroy');
    Route::post('flows/{flow}/edges', [FlowBuilderController::class, 'storeEdge'])->name('flows.edges.store');
    Route::delete('flows/{flow}/edges/{edge}', [FlowBuilderController::class, 'destroyEdge'])->
    >name('flows.edges.destroy');
    Route::post('flows/{flow}/publish', [FlowTemplateController::class, 'publish'])->name('flows.publish');

    Route::get('approvals', [ApprovalCenterController::class, 'index'])->name('approvals.index');
    Route::post('approvals/{approval}/approve', [ApprovalCenterController::class, 'approve'])->
    >name('approvals.approve');
    Route::post('approvals/{approval}/reject', [ApprovalCenterController::class, 'reject'])->name('approvals.reject');

    Route::get('replay', [ReplayConsoleController::class, 'index'])->name('replay.index');
    Route::post('replay/{execution}', [ReplayConsoleController::class, 'replay'])->name('replay.run');

    Route::get('assistant', [AssistantController::class, 'index'])->name('assistant.index');
    Route::post('assistant/threads', [AssistantController::class, 'storeThread'])->name('assistant.threads.store');
    Route::post('assistant/threads/{thread}/messages', [AssistantController::class, 'storeMessage'])->
    >name('assistant.messages.store');

    Route::get('notifications', [NotificationController::class, 'index'])->name('notifications.index');
    Route::post('notifications/{notification}/read', [NotificationController::class, 'markRead'])->
    >name('notifications.read');

```

```
Route::get('infrastructure', [InfrastructureController::class, 'index'])->name('infrastructure.index');
Route::get('intelligence', [ApexIntelligenceController::class, 'index'])->name('intelligence.index');
});
```

## 8.2 Authorization model

Use Laravel gates and policies consistently. Every agent, flow, approval, replay request, and assistant command should be authorization-aware. Super admins can do everything. Operators can manage jobs and replays but should not silently rewrite policy. Approvers can action approvals but not redesign flows. Auditors can inspect everything but cannot mutate records.

```
Gate::define('manage-cockpit', fn (User $user) => $user->hasRole('super_admin') || $user->hasRole('operator'));
Gate::define('approve-high-impact-actions', fn (User $user) => $user->hasRole('super_admin') || $user->hasRole('approver'));
Gate::define('view-audit-and-replay', fn (User $user) => $user->hasAnyRole(['super_admin', 'operator', 'auditor']));
```

## 9. Livewire and Vue Component Map

The UI should default to Livewire for data-heavy administrative screens and use Vue selectively for the Hannah Flow builder canvas and any graphically complex visualizations. This keeps the application coherent, fast to build, and aligned with Laravel's current starter kit and frontend guidance [R3][R4].

| Component            | Recommended tech                | Reason                                               |
|----------------------|---------------------------------|------------------------------------------------------|
| dashboard-overview   | Livewire                        | Server-driven metrics and filters suit Livewire well |
| agent-registry-table | Livewire                        | CRUD-heavy tables, bulk actions, modals              |
| agent-edit-drawer    | Livewire                        | Form validation and server-side authorization        |
| flow-builder-canvas  | Vue                             | Drag/drop graph editing and node positioning         |
| approval-inbox       | Livewire                        | Action-heavy but not graphically complex             |
| replay-console       | Livewire + small Vue visualizer | Mostly server-driven with                            |

|                     |                                       |                                               |
|---------------------|---------------------------------------|-----------------------------------------------|
|                     |                                       | optional playback widgets                     |
| assistant-panel     | Livewire + Alpine or Vue small widget | Chat thread, quick actions, websocket updates |
| notification-center | Livewire                              | List and action semantics map well to Laravel |

## 9.1 Livewire component tree

```

App/Livewire/Cockpit/
DashboardOverview.php
QueueStatusPanel.php
NodeHealthPanel.php
RecentActivityFeed.php
AgentRegistryTable.php
AgentEditDrawer.php
AgentTaskManager.php
ApprovalInbox.php
ReplayConsole.php
NotificationCenter.php
AssistantPanel.php
ApexIntelligenceSummary.php

```

## 9.2 Vue builder component tree

```

resources/js/cockpit/flow-builder/
App.vue
BuilderCanvas.vue
StageLane.vue
FlowNode.vue
EdgeLayer.vue
RightConfigPanel.vue
LeftPalette.vue
PublishRulesPanel.vue
VersionSwitcher.vue
ExecutionPreviewDrawer.vue

```

## 9.3 State model for the builder

```
interface FlowBuilderState {
    flow: FlowTemplate;
    nodes: FlowNode[];
    edges: FlowEdge[];
    selectedNodeId?: string;
    selectedEdgeId?: string;
    unsavedChanges: boolean;
    validationErrors: Record<string, string[]>;
    publishRules: PublishRules;
    executionSettings: ExecutionSettings;
}
```

## 10. Agent Registry CRUD System

The agent registry is the workforce management system for the platform. It is where the super admin or delegated operators define what agents exist, what they are allowed to do, which queues they may service, which models they prefer, what approval policies they must honor, and how they are measured.

### 10.1 Create flow

1. Open Agent Registry and choose Create Agent.
2. Set identity fields: name, slug, product scope, role, runtime type.
3. Set provider and default model, or mark as Laravel-managed if model-free.
4. Bind allowed queue families and allowed job types.
5. Select instruction profile, guardrail profile, approval policy, and secret scope.
6. Set runtime guardrails such as max runtime, token cap, artifact size cap.
7. Save agent and optionally create one or more tasks immediately.

### 10.2 Update flow

All changes should be version-audited. Update operations should write an audit event describing who changed what and whether the change affected runtime permissions, models, tools, or approval requirements.

### 10.3 Archive / deactivate flow

Agents should rarely be hard-deleted. Use deactivate or archive semantics. Any in-flight execution that depends on the agent should either continue with the existing snapshot or fail safely according to policy.

### 10.4 CRUD validation rules

```

public function rules(): array
{
    return [
        'name' => ['required', 'string', 'max:120'],
        'slug' => ['required', 'alpha_dash', Rule::unique('agent_definitions', 'slug')->ignore($this->agentId)],
        'product_scope' => ['required', Rule::in(['hannah', 'atlas', 'apex_intelligence', 'spidernet', 'shared'])],
        'role' => ['required', 'string', 'max:100'],
        'runtime_type' => ['required', Rule::in(['laravel_managed', 'python_service', 'sandboxed_worker',
'inference_proxy'])],
        'allowed_job_types' => ['nullable', 'array'],
        'allowed_queues' => ['nullable', 'array'],
        'tool_allowlist' => ['nullable', 'array'],
        'max_runtime_seconds' => ['nullable', 'integer', 'min:1', 'max:86400'],
        'max_token_budget' => ['nullable', 'integer', 'min:1'],
    ];
}

```

## 10.5 Service layer for the registry

```

class AgentRegistryService
{
    public function create(array $data, User $actor): AgentDefinition {}
    public function update(AgentDefinition $agent, array $data, User $actor): AgentDefinition {}
    public function archive(AgentDefinition $agent, User $actor): void {}
    public function attachTask(AgentDefinition $agent, array $taskData, User $actor): AgentTask {}
    public function rotateModel(AgentDefinition $agent, string $newModel, User $actor): AgentDefinition {}
    public function setStatus(AgentDefinition $agent, string $status, User $actor): AgentDefinition {}
}

```

# 11. Hannah Flow Builder Drag-and-Drop Schema and Execution Mapping

The builder must produce a canonical graph payload. The UI is a convenience layer; the JSON definition is the contract that the backend executes. The recommended shape is a stage-aware graph with node definitions, edge definitions, flow-level settings, and publish/approval rules.

## 11.1 Canonical JSON schema

```

{

```

```

"flow": {
  "id": "flow_01",
  "slug": "q4-launch-campaign",
  "product_scope": "hannah",
  "version": 3,
  "status": "draft"
},
"settings": {
  "parallel_limit": 5,
  "retry_profile": "standard",
  "approval_required": true,
  "timeout_minutes": 30
},
"stages": ["strategist", "creator", "producer", "quality_control"],
"nodes": [
  {"id": "n_start", "type": "start", "stage": null, "config": {}},
  {"id": "n_brief", "type": "agent", "stage": "strategist", "agent_slug": "brief-intake-agent", "config": {"task_key":
"normalize_brief"}},
  {"id": "n_research", "type": "agent", "stage": "strategist", "agent_slug": "research-agent", "config": {"task_key":
"crawl_offer_page"}},
  {"id": "n_copy", "type": "agent", "stage": "creator", "agent_slug": "copy-agent", "config": {"task_key":
"generate_variants"}},
  {"id": "n_approval", "type": "approval", "stage": "creator", "config": {"policy": "publish-risk-medium"}},
  {"id": "n_render", "type": "agent", "stage": "producer", "agent_slug": "render-agent", "config": {"task_key":
"assemble_media"}},
  {"id": "n_qc", "type": "agent", "stage": "quality_control", "agent_slug": "qc-governance-agent", "config":
{"task_key": "rank_and_validate"}},
  {"id": "n_end", "type": "end", "stage": null, "config": {}}
],
"edges": [
  {"from": "n_start", "to": "n_brief", "type": "default"},
  {"from": "n_brief", "to": "n_research", "type": "default"},
  {"from": "n_research", "to": "n_copy", "type": "default"},
  {"from": "n_copy", "to": "n_approval", "type": "default"},
  {"from": "n_approval", "to": "n_render", "type": "approved"},
  {"from": "n_render", "to": "n_qc", "type": "default"},
  {"from": "n_qc", "to": "n_end", "type": "passed"}
]
}

```

## 11.2 Builder node types

| Node type   | Purpose               | Special validation                  |
|-------------|-----------------------|-------------------------------------|
| start / end | Graph boundaries      | Exactly one start, at least one end |
| agent       | Execute an agent task | Must reference valid agent and task |

|           |                              |                                            |
|-----------|------------------------------|--------------------------------------------|
| condition | Branch by data or score      | Must declare expression and outputs        |
| approval  | Pause for human approval     | Must bind approval policy                  |
| parallel  | Fan-out logical grouping     | Must bind child nodes or branch IDs        |
| delay     | Timed wait or schedule       | Must define delay duration or trigger time |
| webhook   | External callback or trigger | Must define endpoint or event binding      |

### 11.3 Execution mapping

At publish time, the backend compiles the flow JSON into executable stage and job templates. Nodes in the strategist stage become stage-specific jobs and dependencies. Approval nodes become durable waiting states. Parallel nodes become queue batch fan-out operations. QC nodes can emit targeted regeneration instructions rather than force a full rerun.

## 12. Cockpit Assistant Backend Logic and Notification System

The cockpit assistant should be implemented as an operations-oriented agent, not as a generic chat endpoint. Its job is to convert telemetry into action. It consumes event and metric streams, produces concise summaries, raises notifications, and supports a bounded set of commands. It should inherit the super admin context only when the authenticated user has the necessary permission.

### 12.1 Responsibilities

- Generate morning brief and periodic summary cards
- Alert on approvals waiting beyond threshold
- Alert on anomaly or KPI deterioration
- Answer natural-language operational questions using the event store and materialized metrics
- Suggest next-best actions with direct deep links into the cockpit
- Dispatch bounded commands such as running a health check or opening a replay view

### 12.2 Core services

```
class CockpitAssistantService
{
    public function summarizeForUser(User $user): array {}
    public function handlePrompt(User $user, string $threadId, string $message): AssistantResponse {}
    public function generateNotifications(): void {}
    public function suggestActions(User $user): array {}
}
```

```

}

class NotificationOrchestrator
{
    public function dispatch(NotificationData $notification): void {}
    public function routeToChannels(NotificationData $notification): void {}
}

class CockpitContextBuilder
{
    public function build(User $user): array {}
}

```

### 12.3 Command handling approach

Use intent classification before execution. The assistant should separate informational queries from action requests. Informational queries can answer directly from summarized metrics or replay data. Action requests such as approve, rerun, open, pause agent, or run health check must be authorization-checked and, for risky actions, turned into approval-gated tasks rather than executed inline.

### 12.4 Notification types

| Type              | Examples                                                      | Default channel            |
|-------------------|---------------------------------------------------------------|----------------------------|
| Approval required | Publishing blocked; privileged code execution requested       | UI + email; WhatsApp later |
| Anomaly           | Conversion drop, queue failure spike, node health degradation | UI + email                 |
| Execution failure | Hannah QC failed twice; Atlas patch build failed              | UI                         |
| Daily summary     | Morning brief, yesterday performance, cost summary            | UI + email optional        |
| Recommendation    | Investigate campaign X, rotate model Y, retry worker pool     | UI                         |

### 12.5 Example assistant response format

```

{
    "message": "Three campaigns are waiting for approval, one anomaly was detected in Hannah Flow, and Atlas has one patch ready for review.",
    "cards": [

```

```

{"type": "approval_list", "items": [...]},
{"type": "anomaly_card", "severity": "high", "execution_id": "exec_123"},
{"type": "quick_actions", "items": ["Review approvals", "Open anomaly", "Show failed jobs"]}
],
"references": ["approval:884", "execution:123", "atlas_patch:93"]
}

```

## 13. Redis, Horizon, Database Plan, Environment Matrix, and Deployment Notes

### 13.1 Horizon assignments

```

'production' => [
  'supervisor-hannah-control' => [
    'connection' => 'redis',
    'queue' => ['hannah-orchestrator', 'hannah-stage-control', 'maintenance-schedules', 'maintenance-retries'],
    'balance' => 'auto',
    'maxProcesses' => 8,
    'tries' => 2,
    'timeout' => 180,
  ],
  'supervisor-hannah-strategy' => [
    'connection' => 'redis',
    'queue' => ['hannah-strategy-brief', 'hannah-strategy-audience', 'hannah-strategy-offer', 'hannah-strategy-plan'],
    'balance' => 'auto',
    'maxProcesses' => 12,
    'tries' => 3,
    'timeout' => 240,
  ],
  'supervisor-hannah-qc' => [
    'connection' => 'redis',
    'queue' => ['hannah-qc-brand', 'hannah-qc-policy', 'hannah-qc-source-truth', 'hannah-qc-ranking'],
    'balance' => 'auto',
    'maxProcesses' => 10,
    'tries' => 1,
    'timeout' => 180,
  ],
  'supervisor-atlas-control' => [
    'connection' => 'redis',
    'queue' => ['atlas-plan', 'atlas-static-review', 'atlas-debug'],
    'balance' => 'auto',
    'maxProcesses' => 8,
  ],
]

```

```

    'tries' => 2,
    'timeout' => 300,
  ],
  'supervisor-apex-intelligence' => [
    'connection' => 'redis',
    'queue' => ['apex-briefing', 'apex-anomaly-detection', 'apex-kpi-rollups'],
    'balance' => 'auto',
    'maxProcesses' => 4,
    'tries' => 2,
    'timeout' => 180,
  ],
];

```

## 13.2 Database/schema plan

| Schema or domain  | Contents                                                                  |
|-------------------|---------------------------------------------------------------------------|
| hannah_core       | requests, executions, stages, jobs, artifacts, publish targets, approvals |
| agent_system      | agent_definitions, tasks, runs, instruction_profiles, guardrail_profiles  |
| event_store       | execution_events, replay_sessions, projections                            |
| assistant         | threads, messages, notifications                                          |
| atlas_core        | repos, code_runs, patches, code review artifacts                          |
| apex_intelligence | kpi summaries, anomalies, recommendations                                 |
| spidernet_core    | tenants, client workflows, connectors, policy packs                       |

## 13.3 Environment variable matrix

```

APP_ENV=production
APP_URL=https://admin.boaai.spidernetos.com
DB_CONNECTION=pgsql
DB_HOST=89.167.118.233
DB_PORT=5432
DB_DATABASE=apex_platform
DB_USERNAME=apex_platform
DB_PASSWORD=***
REDIS_HOST=89.167.118.233
REDIS_PORT=6379
BROADCAST_DRIVER=redis

```

```
QUEUE_CONNECTION=redis
CACHE_STORE=redis
ACTION_API_URL=http://37.27.185.62:8000
ATLAS_API_URL=http://5.223.49.137:9000
INFERENCE_API_URL=http://inference.internal:7000
INTERNAL_API_SECRET=***
OPENAI_API_KEY=***
FAL_API_KEY=***
ELEVENLABS_API_KEY=***
FIRECRAWL_API_KEY=***
ENABLE_FIRECRAWL=true
ENABLE_APPROVALS=true
ENABLE_LOCAL_MODELS=false
```

## 13.4 Internal service URL map

| Service                  | Base URL                                                          |
|--------------------------|-------------------------------------------------------------------|
| Control / Cockpit API    | https://admin.boaai.spidernetos.com or internal app URL on Node 1 |
| Action API               | http://37.27.185.62:8000                                          |
| Atlas API                | http://5.223.49.137:9000                                          |
| Future inference gateway | http://inference.internal:7000                                    |
| Redis                    | redis://89.167.118.233:6379                                       |
| PostgreSQL               | postgres://89.167.118.233:5432/apex_platform                      |

## 14. Phased Build Order

### 14.1 Phase 1 - Hannah production spine

- Deploy cockpit shell, permissions, dashboard, notifications, agent registry, and Hannah execution model
- Implement the four canonical stages: Strategist, Creator, Producer, Quality Control
- Connect Firecrawl-based research enrichment and source-truth validation paths
- Implement approvals center and publish-prep controls
- Deliver the first useful cockpit assistant summary and alert experience

### 14.2 Phase 2 - Atlas platform

- Stand up Atlas API on Node 3

- Add repo ingestion, code planning, codegen/testgen/review workers
- Integrate Atlas into the same agent registry and approval system
- Keep production deployment behind approval gates

### 14.3 Phase 3 - Apex Intelligence

- Materialize KPI rollups and anomaly detection surfaces
- Add executive brief generation and supervisory dashboards
- Expand cockpit assistant context with cross-product intelligence

### 14.4 Phase 4 - SpiderNet externalization

- Introduce tenant-aware templates, client policy packs, client workflows, and metering
- Expose selected cockpit surfaces to client admins under strict scope boundaries
- Use proven Hannah and Atlas patterns rather than inventing net-new runtime logic

### 14.5 Phase 5 - GPU inference plane

Add the GPU node after the core control surfaces and governance systems are stable. The value of local inference is much higher once the workflow, approval, and replay logic are already sound.

## 15. Security, Approvals, Replay, Testing, and Acceptance Criteria

### 15.1 Non-negotiable security rules

- Python services never become the source of truth for execution state.
- No direct DB writes from external worker services except through explicitly sanctioned internal APIs or carefully scoped service accounts.
- Production publishing remains approval-gated until trust thresholds are proven.
- Atlas cannot deploy directly to production by default.
- All high-impact actions write a durable audit and event record.
- Third-party skills, packages, or agent tools are treated as untrusted until reviewed and pinned.
- NemoClaw/OpenShell patterns may inform staged experimentation, but production trust remains with your own scoped-permission and approval system while NemoClaw is alpha [R10] [R11].

### 15.2 Event replay

Every execution should be replayable. The replay console should let the super admin inspect the original request, stage and job transitions, tool invocations, outputs, approvals, and failure points. Replay does not necessarily mean re-execute; it means reconstruct the history faithfully and optionally fork a new run from a chosen safe point.

### 15.3 Feature-test acceptance criteria

1. A super admin can create an agent, attach tasks, and see the agent in the registry.

2. A super admin can build a Hannah template in the builder, save it, validate it, and publish a version.
3. A Hannah execution persists execution, stage, job, artifact, and event records correctly.
4. An approval node pauses execution and resumes only after explicit approval.
5. A cockpit notification is created when a high-severity anomaly or approval event occurs.
6. The cockpit assistant can answer a bounded question such as "What needs my attention?" using real summarized system context.
7. Replay can reconstruct an execution history and present it to an authorized user.

## 15.4 Suggested PHPUnit feature tests

```
it('creates an agent through the registry', function () {
    $user = User::factory()->superAdmin()->create();
    $this->actingAs($user)
        ->post(route('cockpit.agents.store'), [...])
        ->assertRedirect();

    expect(AgentDefinition::query()->where('slug', 'hannah-router')->exists())->toBeTrue();
});

it('saves a flow template graph', function () {
    $user = User::factory()->superAdmin()->create();
    $flow = FlowTemplate::factory()->create();

    $this->actingAs($user)
        ->post(route('cockpit.flows.nodes.store', $flow), [...])
        ->assertStatus(200);
});
```

## 16. Source References and Implementation Notes

The architecture and implementation choices in this document are grounded in official product and framework guidance, then adapted to the concrete constraints of your current three-node Hetzner environment and the target Apex operating model.

- [R1] OpenAI, “A practical guide to building agents,” official guide and PDF.
- [R2] Laravel documentation, queues.
- [R3] Laravel documentation, Horizon.
- [R4] Laravel documentation, scheduling.
- [R5] Laravel frontend and starter kit documentation, including Livewire support.

- [R6] Hetzner Cloud General Purpose / dedicated-resource positioning for sustained production workloads.
- [R7] Hetzner GPU server positioning for AI projects.
- [R8] Google AI for Developers, Gemma 4 model overview and model card.
- [R9] Google AI for Developers, Gemma 4 function-calling documentation.
- [R10] NVIDIA NemoClaw official documentation overview noting alpha / not production-ready.
- [R11] NVIDIA NemoClaw quickstart and GitHub repository noting early preview / do not use in production.
- [R12] Firecrawl official repository and documentation describing crawl/scrape/extract/web-intelligence capabilities.

For the production build, the next logical deliverables after this document are: (a) the Laravel migration pack; (b) the Livewire components and Vue builder shell; (c) seeded system agents and Hannah templates; (d) the first action-plane service adapters; and (e) the first end-to-end feature tests that prove the super-admin cockpit can control the platform safely.

## Appendix A - Recommended Folder Structure

```

app/
  Http/Controllers/Cockpit/
    CockpitDashboardController.php
    AgentRegistryController.php
    AgentTaskController.php
    AgentPerformanceController.php
    FlowTemplateController.php
    FlowBuilderController.php
    ApprovalCenterController.php
    ReplayConsoleController.php
    AssistantController.php
    NotificationController.php
    InfrastructureController.php
    ApexIntelligenceController.php
  Livewire/Cockpit/
    DashboardOverview.php
    AgentRegistryTable.php
    AgentEditDrawer.php
    ApprovalInbox.php
    ReplayConsole.php
    AssistantPanel.php
  Models/
    AgentDefinition.php
    AgentTask.php
    AgentRun.php
    FlowTemplate.php
    FlowNode.php

```

```
FlowEdge.php
Notification.php
AssistantThread.php
AssistantMessage.php
resources/js/cockpit/flow-builder/
App.vue
BuilderCanvas.vue
FlowNode.vue
RightConfigPanel.vue
database/migrations/
... cockpit and flow builder migrations ...
```

## Appendix B - Super-Admin Daily Operating Checklist

1. Open Overview and review node health, queue failures, pending approvals, and overnight anomalies.
2. Open assistant briefing and inspect the top three recommended actions.
3. Clear approval backlog that affects production publishing or high-impact changes.
4. Review any failed Hannah or Atlas executions that crossed severity threshold.
5. Review queue saturation or node degradation and trigger a health check when needed.
6. Review agent registry changes and ensure no unauthorized or risky updates were applied.
7. Close the day with a summary of costs, campaign outcomes, engineering outputs, and outstanding risks.